On the Truth of Interaction

Contextual Compute: Design and Implementation

PIC: Operational Semantics of Participant-Indexed Computation

Anantha Krishnan
Sarva Labs Princeton, NJ, USA
ak@sarva.ai

March 2026

Abstract

This paper derives the operational semantics of Contextual Compute from the K-Machine's foundational requirement that *who* is a fundamental dimension of computation. We formalise the Participant–Interaction–Context (PIC) model: the three irreducible semantic primitives whose presence, and whose categorical structure, force a machine in which consensus, ordering, and finality are participant-anchored and context-scoped — dissolving the need for anonymous global state, network-wide ordering, and external agreement.

We define participant semantics (sovereignty, persistence, the 4-tuple $p = (id, \sigma, f_p, H)$), Context Superstate semantics (the seven sub-contexts, the anchor chain, the Participant-Indexed State Completeness Theorem, value objects $v = \langle p, c, x \rangle$), interaction semantics (atomicity, linearity, contextual scope, determinism, intrinsic finality), and witness semantics (the full constituent set $\mathcal{C}_\iota = P_\iota \cup W_\iota$, the Value–Agreement Equivalence, the dissolution of the Cartesian consensus recursion).

Six axioms — Conservation, Consent, Linearity, Witness Sufficiency, Composability, and Commutativity — are shown to be necessary consequences of the K-Machine's structure. Together they define the PIC value category V_{PIC}: a symmetric monoidal category without a diagonal morphism. Participant-indexed contextual state strictly subsumes anonymous global state: $\mathbf{C}_{\mathrm{Cartesian}} \subsetneq V_{\mathrm{PIC}}$. Classical computation is the degenerate case at WHO $= 0$.

The paper closes with a derivation chain tracing each formal result to its categorical source, and a correspondence table mapping PIC's semantic objects to their realisations in the Contextual Compute Protocol.

Contents

1 Introduction **4**

2 Related Work **6**
2.1 Linear Logic and Linear Type Systems 6
2.2 Categorical Quantum Mechanics 6
2.3 Distributed Ledgers and Consensus 6
2.4 Linear Resources in Blockchain Systems 7

2.5 Summary of Positioning . . . 7

3 The Semantic Contract **7**
3.1 What This Paper Derives From . . . 7
3.2 The Demand for Operational Semantics . . . 9
3.3 PIC Is Forced, Not Chosen . . . 9
3.4 The Five Derived Properties . . . 9
3.5 Structure of This Paper . . . 10

4 Participant Semantics **10**
4.1 The PIC Triple . . . 10
4.2 WHO as a Computational Primitive . . . 10
4.3 Formal Definition . . . 10
4.4 Sovereignty . . . 11
4.5 Participant Persistence . . . 11
4.6 Any Entity Is a Participant . . . 11
4.7 What the Participant Is Not . . . 11
4.8 What PIC Excludes . . . 11

5 Context Superstate Semantics **12**
5.1 From Abstract Context to Concrete Superstate . . . 12
5.2 Participant-Indexed State Completeness . . . 12
5.3 Value Objects . . . 13
5.4 The Seven Sub-contexts . . . 14
5.5 Trust as the t-Dimension . . . 15
5.6 The Anchoring Topology . . . 15
5.7 The Entanglement Structure . . . 15
5.8 Context Inheritance and Shared Spaces . . . 15
5.9 Context Height and Linearity . . . 16
5.10 Context as the Semantic Unit of Value . . . 16
5.11 The Only Mechanism by Which Contexts Change . . . 16
5.12 Formal Definition . . . 17
5.13 Four Structural Properties . . . 17
5.14 Consent as a Semantic Primitive . . . 17
5.15 The Interaction Context Space . . . 18
5.16 Interactions Are Not a Sequence . . . 18
5.17 Finality . . . 18
5.18 What Interaction Excludes . . . 19
5.19 The Complexity Consequence . . . 19
5.20 Agreement Is Semantic, Not Procedural . . . 19
5.21 The Categorical Distinction . . . 20
5.22 Witnesses Are Statefully Embedded in Trust . . . 20
5.23 The Full Constituent Set . . . 20
5.24 Witness Set Formal Specification . . . 21
5.25 Witnesses Across Inherited Contexts . . . 21
5.26 The Value–Agreement Equivalence . . . 21
5.27 Derived Corollaries . . . 21
5.28 Why Witnesses Replace Consensus: The Cartesian Recursion . . . 22
5.28.1 The Recursion . . . 22
5.28.2 The PIC Resolution: Participant as Definitive Anchor . . . 22

5.28.3 Why Consensus Cannot Be Added Back . . . 23
5.28.4 The Structural Statement . . . 23
5.29 Closing the Loop . . . 24

6 Network Objects: The First Linear Primitive at Network Level 24
6.1 The Semantic Gap . . . 24
6.2 The Distinction . . . 24
6.3 The Significance . . . 25
6.4 Scope . . . 25

7 The Six Axioms and PIC as Category 25
7.1 Axioms as Consequences . . . 25
7.2 The Six Axioms . . . 25
7.2.1 The Causal Record . . . 27
7.2.2 The Partial Order Objection . . . 27
7.2.3 Commutativity Within Shared Inherited Contexts . . . 27
7.3 Derived Properties . . . 28
7.4 PIC as a Category . . . 28
7.5 What PIC Eliminates . . . 30

8 The Derivation Chain and Bridge to Implementation 31
8.1 The Architecture as Categorical Entailment . . . 31
8.2 What PIC Does Not Specify . . . 31
8.3 The Bridge to Implementation . . . 33
8.4 The Remaining Frontier . . . 33
8.5 Closing Statement . . . 34

1. Introduction

Every computation since Turing has operated over an anonymous global state. The tape has no owner. The symbol has no participant. The machine proceeds independently of who is involved. This is the correct model for information processing: information is observer-independent, freely copyable, and combinable without loss. Shannon's theory [15] formalises this precisely. Classical computation, distributed systems, and databases are all correctly optimised for this domain.

Value is none of these things.

Value is participant-anchored: the same nominal quantity held by different participants under different conditions does not represent the same value. Value is linear: it cannot be copied without creating the conditions for conflict. Value is contextual: it exists only within or across participant-owned state spaces, and resolves only through participant-constituted interaction. These are not engineering constraints. They are ontological facts about the structure of value.

The consequence of ignoring these facts is a structural category error that has propagated through every value-bearing distributed system ever built. When value is treated as a special case of information, it must be managed on a Cartesian substrate — one equipped with the diagonal morphism that permits copying. Copying produces multiple equally valid representations. Multiple representations require external arbitration. External arbitration requires global ordering. Global ordering requires consensus. Consensus requires a reference state. But the reference state is what consensus is trying to establish. The circle is permanent. No increase in speed, stake, or cryptographic sophistication can remove it, because the circle is structural, not incidental.

This paper addresses the question: what is the correct operational semantics for value computation?

The foundational answer is the separation of state space from computational space. In every model since Turing — single machine or distributed — state is anonymous. It has nowhere to live except the computational space, so every computation that needs compute also needs to access shared state, and ordering follows necessarily. This is true on a single machine and acutely visible in distributed systems: consensus, global ordering, and replication all arise from the same root condition — state is anonymous and lives in the computational space. Contextual Compute introduces WHO as a primitive dimension of computation alongside WHAT, WHERE, and HOW. State becomes participant-indexed — it lives with the participant, not in the computational space. Shared computational spaces remain: a market, a protocol, a governance context are each participants with their own Logic, Prefs, and state. But that state is participant-indexed to them. No anonymous computational substrate exists. Ordering is structurally undefined not because spaces have no state, but because no state exists outside a participant's context. This is not an optimisation of existing systems. It is what computation looks like when WHO exists.

Contributions

We make the following contributions:

1. **The PIC model.** We define the Participant–Interaction–Context (PIC) triple (P, I, C) as the operational semantics of the K-Machine — the abstract machine for value computation introduced in the companion paper [10]. We show that each of the three

primitives is irreducible: removing any one collapses the model back into classical information processing.

2. **Participant semantics.** We formalise participants as 4-tuples $p = (id, \sigma, f_p, H)$ with sovereign, persistent, participant-indexed state. We establish Property 2.1 (Sovereignty) as a structural consequence of the absence of global state, not a policy-enforced property.
3. **Context Superstate semantics.** We define the Context Superstate $C(p)$ as a seven-sub-context structure anchored by $\mathrm{Trust}(p)$, and prove the Participant-Indexed State Completeness Theorem (Theorem 5.1): anonymous global state — state that exists independently of participants and is aggregated across them — is structurally unavailable in V_{PIC}. The complete and authoritative network state is $\Sigma = \{C(p)\}_{p \in P}$, which is participant-indexed and requires no consensus to be authoritative.
4. **Interaction semantics.** We formalise interactions as 5-tuples $\iota = (S, T, W, f_\iota, \mathrm{Auth}_\iota)$ and establish four structural properties — atomicity, linearity, locality, and determinism — together with intrinsic finality and the Interaction Context Space (ICS).
5. **Witness semantics.** We formalise witnesses as constituents rather than validators, define the full constituent set $\mathcal{C}_\iota = P_\iota \cup W_\iota$, and prove the Value–Agreement Equivalence (Theorem 5.11): value exists if and only if all constituents participate. We dissolve the Cartesian consensus recursion by showing it has no base case on a Cartesian substrate, while PIC's participant-anchored linear state provides that base case by construction.
6. **Six axioms and the PIC value category.** We derive six axioms — Conservation, Consent, Linearity, Witness Sufficiency, Composability, and Commutativity — as necessary consequences of the K-Machine's structure, and use them to define V_{PIC}: a symmetric monoidal category without a diagonal morphism. We establish the strict inclusion $\mathbf{C}_{\mathrm{Cartesian}} \subsetneq V_{\mathrm{PIC}}$ and show that classical computation is the degenerate case at $\mathrm{WHO} = 0$.
7. **Network Objects.** We introduce Network Objects as the first computer science primitive designed for linear types in distributed systems — the realisation of a linear type at network level.

Positioning

This paper is CC4 in the Contextual Compute Series (*On the Truth of Interaction*). CC1 [12] establishes the philosophical and mathematical foundation of participant-indexed value through the Quantum Value Paper. CC2 [11] establishes the Domain Separation Theorem: value and information are objects in categorically distinct monoidal categories and no structure-preserving functor maps one into the other. CC3 [10] introduces the K-Machine: the abstract machine that formalises Contextual Compute by placing the participant as a primitive dimension of computation alongside WHAT, WHERE, and HOW, and proves that anonymous global computation is recovered exactly at $\mathrm{WHO} = 0$. The present paper derives the operational semantics of the K-Machine through the PIC model. CC5 (forthcoming, Sarva Labs) specifies the Contextual Compute Protocol: IFP, CP, ISM, and Context Power as a normative standard for any CC-compliant implementation.

Structure

Section 2 surveys related work. Section 3 states the semantic contract — what this paper derives from and what it derives. Section 4 defines participant semantics. Section 5 defines Context Superstate semantics. Section 5.10 defines interaction semantics. Section 5.19

defines witness semantics. Section 6 introduces Network Objects. Section 7 states the six axioms and establishes V_{PIC} as a category. Section 8 closes the derivation chain and maps PIC to its implementation.

2. Related Work

2.1 Linear Logic and Linear Type Systems

The foundational treatment of linearity in computation is Girard's linear logic [9], which distinguishes resources that may be used exactly once from those that may be freely copied or discarded. Wadler [17] introduced linear types into programming languages to enforce single-use resource discipline. Walker [18] provides a comprehensive treatment of substructural type systems in the context of programming language theory, and Mazurak et al. [13] give a lightweight account of linear types in System F.

PIC is related to, but distinct from, this body of work. Linear type systems operate at the *language* level: they enforce linearity as a property of program expressions within a classical computational substrate. PIC operates at the *machine* level: linearity is a property of the state space itself, not of the language operating over it. The K-Machine's state space has no diagonal morphism; there is no substrate from which copying could be done even in principle. This is a stronger result — linearity by structure rather than linearity by enforcement.

2.2 Categorical Quantum Mechanics

Abramsky and Coecke [1] introduced categorical quantum mechanics, showing that quantum protocols can be given a compositional semantics in compact closed categories equipped with a symmetric monoidal structure. Coecke and Kissinger [8] extended this to multipartite entanglement. Baez and Stay [2] exhibit a deep correspondence between physics, topology, logic, and computation via monoidal categories.

This programme is the closest categorical relative of the present work. V_{PIC} is a symmetric monoidal category without a diagonal morphism — a structure that explicitly parallels quantum state spaces, where the no-cloning theorem prohibits the diagonal morphism. The companion paper [11] makes this parallel precise and categorical: it is not an analogy but the same categorical move — replacing anonymous state spaces with participant-indexed ones. The present paper works out the full operational semantics of a machine built in this category.

2.3 Distributed Ledgers and Consensus

Nakamoto [14] introduced the first practical Byzantine-fault-tolerant distributed ledger using proof-of-work consensus. Buterin [5] extended this to a general-purpose smart contract platform. Castro and Liskov's PBFT [6] provides the foundational practical BFT protocol that underlies most permissioned and proof-of-stake systems.

PIC's witness model achieves the same f-Byzantine fault tolerance threshold as classical BFT ($2f+1$ of $3f+1$) without requiring replicated global state. The structural difference is categorical: consensus-based systems operate in $\mathbf{C}_{\mathrm{Cartesian}}$ and require consensus because copying creates multiple equally valid representations; PIC operates in V_{PIC} where no copies exist, so the question consensus answers — which copy is correct? — cannot be asked.

2.4 Linear Resources in Blockchain Systems

The Move language [3], developed for the Libra/Diem project and subsequently adopted in Sui and Aptos, introduces linear resource types at the smart contract language level: resources cannot be copied or discarded, only moved between storage locations. The extended UTxO model [7] extends Bitcoin's UTxO accounting with programmable scripts, preserving the linear accounting property of unspent outputs.

Sui's object model [4] achieves partial commutativity by distinguishing owned objects (accessible to one account) from shared objects (globally accessible), permitting parallel execution over owned objects while serialising over shared ones. FastPay [16] achieves high-throughput settlement for payments over disjoint accounts without global consensus.

PIC generalises beyond all of these in a specific categorical sense. Move enforces linearity at the language level over a Cartesian substrate; the substrate remains copyable even if the language prohibits it. Sui's commutativity is partial — it fails precisely at shared objects, which are common in multi-party applications. PIC's commutativity is structural and holds even within shared inherited contexts, because shared contexts in PIC carry shared *computation* but not shared *value state*. Value remains in $\text{Assets}(p_i)$ for each participant p_i. There is no shared value pool for operations to contend over.

2.5 Summary of Positioning

PIC occupies a position not previously filled in the literature. Linear type systems enforce linearity at the language level on a Cartesian substrate. Blockchain consensus systems enforce linearity via global ordering on a copyable substrate. Both leave the diagonal morphism in place and manage its consequences by enforcement. PIC removes the diagonal morphism at the machine level — as a property of the state space itself — and derives consensus-freedom, ordering-freedom, and intrinsic finality as structural consequences, not protocol features. The closest categorical relative is quantum mechanics: both operate in symmetric monoidal categories without a diagonal morphism, and both resolve through constituent participation rather than external observer agreement.

3. The Semantic Contract

3.1 What This Paper Derives From

The K-Machine is defined in the companion theory paper as a 6-tuple:

$$K = (P, \Sigma, I, \delta, W, \Phi)$$

where P is the set of participants; $\Sigma = \{\Sigma_p\}_{p \in P}$ is the family of participant-indexed state spaces; I is the set of interactions; δ is the partial transition function; W is the witness assignment function; and Φ is the finality function.

The K-Machine generalises classical state machines by abandoning anonymous global state in favour of participant-indexed linear state. It is not a refinement of the Turing Machine. It belongs to a different computational category. The Turing Machine operates on a global tape whose symbols belong to no one; state exists independently of who accesses it. The K-Machine operates over interactions, not instructions; state exists only within participant-owned contexts; execution is not sequencing over time but transformation of entanglement.

Turing Machine	**K-Machine**
Global tape	Participant-indexed state
Anonymous symbols	Contextual value objects
Sequential execution	Interactional execution
Copyable state	Linear state
External agreement	Constitutive agreement

The K-Machine has no global clock, no scheduler, no instruction pointer, and no total order. Control emerges from participant availability and contextual constraints.

Notation bridge. CC3 [10] defines the K-Machine as a 7-tuple $\mathcal{K} = (\mathbb{P}, \mathbb{C}, \Sigma_{\text{in}}, \Gamma, \psi, p_0, C_0)$ where $\psi : \mathbb{P}\times\mathbb{C}\times\Sigma_{\text{in}} \to \mathbb{P}\times\mathbb{C}\times\mathbb{V}$ is the participant-indexed transition function and Σ_{in} is the input alphabet. PIC adopts an operationally equivalent formulation $K = (P, \Sigma, I, \delta, W, \Phi)$ suited to the semantics layer, with the following correspondence: P corresponds to $\mathbb{P}$ (participant space); $\Sigma = \{\Sigma_p\}_{p\in P}$ is the family of participant-indexed state spaces (equivalent to $\mathbb{C}$, reindexed by participant — note this Σ is distinct from CC3's input alphabet Σ_{in}, and also distinct from the complete network state $\Sigma = \{C(p)\}_{p\in P}$ used later in this paper, which inherits its notation from the K-Machine's participant-indexed structure); I is the set of interactions (operational counterpart of Γ); δ is the K-Machine's partial transition function (operational counterpart of ψ); W is the witness assignment function; and Φ is the finality function. Within PIC, the participant 4-tuple uses f_p for the participant's own local transition function, distinct from δ (the K-Machine's global transition). The interaction 5-tuple uses f_ι for the linear transformation: this corresponds to parallel transport along the geodesic $\gamma : [0, 1] \to M$ in CC3's geometric model [10] — the geodesic between participants p and q is realised operationally as the interaction $\iota = (S, T, W, f_\iota, \text{Auth}_\iota)$, where f_ι is the operational form of parallel transport and $\text{Auth}_\iota = \{\text{sig}(p) : p \in S\}$ is the cryptographic consent gate.

Two theorems from the theory paper govern everything that follows.

The Recovery Theorem establishes that classical computation is the degenerate case of Contextual Compute obtained by setting the participant dimension to the trivial unit object (WHO = 0). The inclusion $\mathbf{C}_{\text{Cartesian}} \subsetneq V_{\text{PIC}}$ is strict: every classical computation is expressible in PIC, but PIC expresses computations that have no classical counterpart.

The Domain Separation Theorem establishes that value and information are objects in categorically distinct monoidal categories. The value category V_{PIC} is symmetric monoidal without a diagonal morphism; the information category is Cartesian monoidal with the diagonal morphism present. No structure-preserving functor maps one into the other.

These two theorems are the source from which all PIC semantics derive.

The categorical and architectural statements are the same statement. $\mathbf{C}_{\text{Cartesian}}$ is the category of anonymous computation: the diagonal morphism is present, copying is free, state has no natural owner, and state space and computational space coincide. V_{PIC} is the category of participant-indexed computation: no diagonal morphism, copying is undefined, state is sovereign, and state space and computational space are separated. Let C_I denote anonymous computation and C_P denote participant-indexed computation. Then:

$$C_I \;=\; \mathbf{C}_{\text{Cartesian}}, \qquad C_P \;=\; V_{\text{PIC}}$$

The strict inclusion $\mathbf{C}_{\text{Cartesian}} \subsetneq V_{\text{PIC}}$ is not merely a mathematical result. It is the precise formal expression of what changes when WHO becomes a primitive dimension of computation. Every ordering requirement, every consensus protocol, every replication mechanism in classical distributed systems is a consequence of C_I — of operating in a category where copying is free and state has no owner. They do not appear in C_P because the morphism structure that produces them does not exist there.

3.2 The Demand for Operational Semantics

The K-Machine defines *where* computation happens: in participant-indexed state spaces. It does not define *how* computation proceeds within those constraints. In classical computation, the Turing Machine is paired with an operational semantics — the λ-calculus, abstract state machines, or equivalent formalisms. For Contextual Compute, this role is played by the PIC model.

PIC is the operational semantics of the K-Machine: it specifies the execution rules governing participant-indexed state, interaction formation, and linear state transformation. PIC is not a protocol and not an implementation strategy. It is the semantics required for any machine that correctly computes value.

3.3 PIC Is Forced, Not Chosen

PIC introduces three irreducible semantic domains:

- **Participants** (P): entities capable of holding value and consenting to interactions.
- **Interactions** (I): bounded execution frames in which contexts transform.
- **Contexts** (C): participant-owned state spaces within which value exists.

None of these domains can be eliminated or represented as metadata. They are semantic primitives. Removing participants collapses value to anonymous content — the Cartesian model. Removing contexts removes the locus in which value exists. Removing interactions removes the only mechanism by which value transforms. Each removal collapses the semantics back into information processing.

PIC is not a refinement of classical operational semantics. It is a different semantic foundation, required by the category of value.

3.4 The Five Derived Properties

From the K-Machine and PIC together, five properties follow as structural consequences:

1. **Sovereign state.** Every participant's context is participant-owned and modifiable only through that participant's explicit consent.
2. **Structural parallelism.** Interactions over disjoint participant contexts commute categorically. Parallelism is structural — a consequence of the absence of shared state.
3. **Intrinsic finality.** An interaction witnessed by sufficient quorum is final at the moment of completion. Finality is constituted by the interaction, not accumulated through confirmation depth.
4. **Complexity independence from network scale.** Agreement is scoped to the Interaction Context Space (ICS), defined by named participants, independent of network size N. The cost of any interaction is $O(k \times t)$ where k is the number of participants and t is interaction complexity.

5. **Native interoperability.** The participant is the interoperability primitive. Cross-context value transfer requires no bridge protocol, no custodian, no wrapped representation. Interoperability is structural.

3.5 Structure of This Paper

Section 4 defines participant semantics. Section 5 defines Context Superstate semantics. Section 5.10 defines interaction semantics. Section 5.19 defines witness semantics. Section 7 states the six axioms and establishes PIC as a category. Section 8 closes the derivation chain and bridges to the implementation layer.

4. Participant Semantics

4.1 The PIC Triple

Definition 4.1 (PIC System)**.** A PIC system consists of a triple (P, I, C) where:

- P is a finite set of participants,
- $C = \{C(p)\}_{p \in P}$ is a family of participant-owned contexts, and
- I is a set of interactions defined over subsets of P.

No element of this triple is optional. Removing any component collapses the semantics back into information processing.

The K-Machine defines where computation happens: in participant-indexed state spaces. PIC defines how computation happens: through linear, scoped interactions. Together they form the minimal computational model required to represent value faithfully, without simulation, enforcement, or global coordination.

4.2 WHO as a Computational Primitive

Classical computation treats participants as data within an anonymous state space. Identity is metadata. The computation proceeds independently of who is involved.

This is correct for information. Information is observer-independent. The same bit string has the same meaning regardless of who holds it. Value is not observer-independent:

$$\text{Value} \neq f(\text{Content}) \qquad \text{Value} \neq f(\text{Content, Metadata})$$

Value requires participant-indexed state spaces $\{\Sigma_p\}_{p \in P}$ where each participant p owns a sovereign context Σ_p, and value exists only within or across these contexts through interaction.

This move mirrors a known transition in physics. Classical mechanics assumed observer-independent state. Quantum mechanics abandoned this assumption and indexed state by measurement context. Contextual Compute makes the analogous move for value. The parallel is not metaphorical. It is the same categorical move: abandoning anonymous state spaces.

4.3 Formal Definition

Definition 4.2 (Participant)**.** A participant $p \in P$ is a 4-tuple:

$$p = (id,\ \sigma,\ f_p,\ H)$$

where:

- *id* is a globally unique, unforgeable identifier,
- σ is the participant's local state — the current Context Superstate $C(p)$,
- δ is the participant's transition function governing how $C(p)$ evolves, and
- H is the participant's interaction history — the sequence of witnessed interactions.

A participant $p \in P$ is not an identity record or address. A participant is a computational locus that owns state and participates in transitions. Each participant p owns exactly one context $\mathrm{ctx}(p) = C(p)$. Ownership is exclusive and non-transferable; a context cannot be read, modified, or duplicated except through an interaction involving its owner.

4.4 Sovereignty

Property 4.3 (Sovereignty)**.** Each participant p has exclusive authority over its context $C(p)$. No transition may modify $C(p)$ without p's participation.

This property is structural, not enforced by policy. It follows from state being participant-indexed: $C(p)$ is owned by p and exists only within participant-indexed structure. No anonymous global state exists from which $C(p)$ could be modified independently of p. The modification is not prevented — it is non-expressible. There is no instruction for it.

4.5 Participant Persistence

A participant is not a session, a key pair, or an address. The participant is persistent, accumulating interaction history, carrying sovereign state across all contexts it inhabits. A participant's state at any moment is fully determined by its interaction history H. Given H, the current Context Superstate $C(p)$ is cryptographically recoverable.

4.6 Any Entity Is a Participant

The participant model is general. A human individual, a corporation, a software agent, or an IoT device — any entity capable of holding value, consenting to interactions, and maintaining a cryptographically anchored state history satisfies Definition 4.2. This generality is not a design choice. It follows from the definition. WHO is a dimension of computation, as general as WHERE, WHAT, and HOW.

4.7 What the Participant Is Not

A *participant* is not a key pair (key pairs are elements of $\mathrm{Keys}(p)$ within $C(p)$; the participant persists through key rotation). It is not an account (accounts are records in an application's storage; the participant exists independently of any application). It is not a session (sessions are time-bounded and stateless; the participant accumulates state continuously).

4.8 What PIC Excludes

PIC deliberately excludes from its operational semantics: global state, replication, total or partial ordering, external consensus, rollback, and external validation. These mechanisms are unnecessary once value is modelled correctly. They arise only when value is misrepresented as information — when the diagonal morphism is present and copying is free. In PIC the diagonal morphism does not exist.

5. Context Superstate Semantics

5.1 From Abstract Context to Concrete Superstate

Bridge from K-Machine geometry to PIC operational semantics. CC3 [10] formalises participant context geometrically: participant p is a point on a smooth Riemannian manifold M, context is the tangent space T_pM equipped with metric tensor g_p, interactions are geodesics $\gamma : [0, 1] \to M$, and value is a scalar field $v : M \to \mathbb{R}$. PIC operationalises this geometric structure. The correspondence is direct: the tangent space T_pM is realised as the Context Superstate $C(p)$; the metric tensor g_p is carried by Trust(p) — specifically, Trust(p) accumulates through interactions in precisely the way the metric tensor evolves along the KCC trajectory τ_p in K-Machine theory, and Intel(p) carries the EPI substrate EPI$(P, t) = g_p(t)$; the anchor chain $\alpha(h)$ is the operational form of the KCC trajectory $\tau_p : \mathbb{R} \to T_pM$, recording the causal succession of context states; and the Krama Value Space $\mathcal{H}_K = \mathcal{H}_x \otimes \mathcal{H}_y \otimes \mathcal{H}_z \otimes \mathcal{H}_p$ is realised through the seven sub-contexts of $C(p)$, with $\mathcal{H}_x$ (WHAT) $\leftrightarrow$ Assets(p), $\mathcal{H}_y$ (WHERE) $\leftrightarrow$ Storage(p), $\mathcal{H}_z$ (HOW) $\leftrightarrow$ Logic(p), and $\mathcal{H}_p$ (WHO) $\leftrightarrow$ {Keys(p), Prefs(p), Intel(p), Trust(p)}. Note also that $\Sigma = \{C(p)\}_{p\in P}$ denotes the complete participant-indexed network state throughout this paper — distinct from the K-Machine's Σ as input alphabet in CC3.

The K-Machine specifies that state is participant-indexed. The complete and authoritative network state is the family:

$$\Sigma = \{C(p) \mid p \in P\}$$

Each context $C(p)$ is owned, linear, and sovereign. This is the correct and complete form of global state — participant-indexed, context-scoped, requiring no consensus to be authoritative. Anonymous global state — a single aggregate Σ_{global} existing independently of participants — is structurally unavailable: no operation constructs it from participant contexts. Formally, there is no total function:

$$\bigotimes_{p\in P} C(p) \longrightarrow \Sigma_{\text{global}}$$

because the tensor product here is not Cartesian: no diagonal or projection morphisms exist.

5.2 Participant-Indexed State Completeness

Theorem 5.1 (Participant-Indexed State Completeness)**.** *In a PIC system, all state is participant-indexed and context-indexed. The complete and authoritative network state is the family $\Sigma = \{C(p)\}_{p\in P}$. Anonymous global state — a state Σ_{global} that exists independently of participants and aggregates across them — is structurally unavailable: V_{PIC} contains no morphism to construct it from participant contexts.*

Proof (Sketch). Assume, for contradiction, that an anonymous aggregate state Σ_{global} exists alongside $\Sigma = \{C(p)\}_{p\in P}$. Then for each participant p there must exist a projection:

$$\pi_p : \Sigma_{\text{global}} \longrightarrow C(p)$$

and for any interaction involving participants $P_\iota \subseteq P$, there must exist an embedding:

$$\iota : \{C(p) \mid p \in P_\iota\} \longrightarrow \Sigma_{\text{global}}$$

Such projections and embeddings require contraction and weakening: contexts must be copyable and discardable. But contexts are linear by construction. No such morphisms exist. Therefore Σ_{global} cannot exist alongside participant-indexed state. The family $\Sigma = \{C(p)\}_{p \in P}$ is already the complete and correctly locus-ed state of the system. □

What this establishes. Participant-indexed contextual state is not a restricted form of global state. It is the complete and superior form. Every state expressible in anonymous global systems is expressible in PIC at WHO = 0. The inclusion $\mathbf{C}_{\text{Cartesian}} \subsetneq V_{\text{PIC}}$ is strict: PIC expresses states and computations that anonymous systems cannot. Within PIC, every state is more precise (anchored to its owner), more authoritative (no consensus required), more parallel (disjoint contexts commute structurally), and more secure (sovereign: no instruction exists to modify it without the participant's participation).

Why replication fails. Replication assumes separability — that the same state can exist independently in multiple locations. For information this is correct; for value it is incoherent. Replicating a balance does not replicate value. It replicates *claims* about value. When two replicas diverge, there is no fact of the matter to reconcile against, because value never existed in replicated form. Consensus does not discover truth. It manufactures consistency.

From replication to interaction. Contextual Compute replaces replication with interaction. Instead of asking "What is the global state?" the model asks "Which participants interacted, and what transformation occurred?" This question is contextual, decidable, and final.

Relation to quantum structure. The absence of global state mirrors the structure of quantum systems. Quantum mechanics does not admit a global, observer-independent state; state is defined relative to measurement context. Similarly, value does not admit a global, participant-independent state; value is defined relative to participant context. This is not an analogy. It is the same categorical move: abandoning anonymous state spaces.

Within each participant's chain, state succession is cryptographically committed:

$$\alpha(h+1) = H\big(id(p) \parallel h{+}1 \parallel \alpha(h) \parallel \sigma(h+1)\big)$$

where $\alpha(h)$ is the anchor at height h, $id(p)$ is the participant's unique identifier, and $\sigma(h{+}1)$ is the new state. Given $\alpha(h)$ and claimed history $(\sigma_0, \ldots, \sigma_h)$, the history can be verified in $O(h)$ hash operations. Any modification to any point in the history changes the current anchor. History is immutable by construction.

5.3 Value Objects

Definition 5.2 (Value Object)**.** A value object is a linear triple:

$$v = \langle p,\, c,\, x \rangle$$

where p is the owning participant, c is the governing context, and $x \in X$ is the content. There is no projection from v to x that preserves value. Removing p or c destroys meaning.

A value object is not content plus metadata. It is an irreducible triple in which participant, context, and content are entangled. A nominal balance as a standalone figure is malformed in PIC — it is a projection that destroys p and c, leaving only x, which is not a value object. The correctly formed expression is: $\text{Assets}(p) \otimes \text{Trust}(p)$.

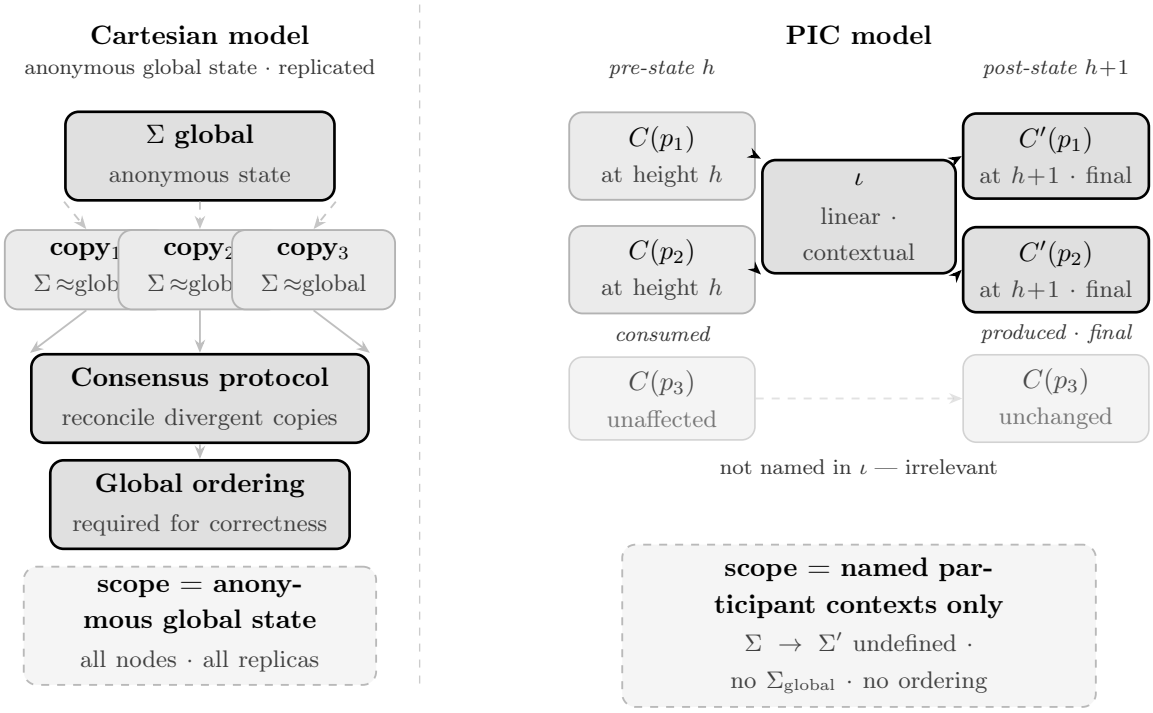

Figure 1: Cartesian vs PIC state space. Left: anonymous global state requires replication, consensus, and global ordering. Right: participant-indexed linear state transforms contextually — scope is defined by named participant contexts, not network topology. $\Sigma \to \Sigma'$ is undefined in PIC.

5.4 The Seven Sub-contexts

The Context Superstate $C(p)$ is the complete state relevant to participant p. PIC derives exactly seven sub-contexts from the K-Machine's four dimensions (WHAT, WHERE, HOW, WHO).

- Assets(p): Linear value objects — tokens, balances (spendable, delegated, locked, mandated), asset metadata. The WHAT dimension.
- Storage(p): Participant-owned persistent data — key-value state, documents, credentials, blob storage. The WHERE dimension.
- Logic(p): Participant-owned computation — interaction handlers, rules, versioned code, immutable when sealed. The HOW dimension.
- Keys(p): Cryptographic material — active key pairs, rotation history, recovery paths, revocation, delegation weights, signature algorithm specifications including classical and post-quantum lattice-based schemes. The WHO dimension's infrastructure layer.
- Prefs(p): Operational policies — compute locus, storage locus, access policies, deny lists, privacy preferences. The WHO dimension's policy layer.
- Intel(p): EPI substrate — participant-specific embeddings, inference outputs anchored to p's interaction history. Formal realisation of $\mathrm{EPI}(P, t) = g_p(t)$ from K-Machine theory. Not generic embeddings — non-transferable, anchored to Trust(p). The WHO dimension's accumulated intelligence layer.
- Trust(p): Witness anchoring — $W(p)$ statefully embedded; inherited contexts (roles

and shared spaces); Trust Quotient; Reputation Index; network behaviour attestations; delegation authorities and mandate chains. The t-dimension made concrete.

5.5 Trust as the t-Dimension

Theorem 5.3 (Trust as t-Dimension)**.** Trust(p) *is the structural anchor of* $C(p)$*. The six remaining sub-contexts derive their participant-indexed character from their entanglement with* Trust(p)*.*

The K-Machine's state space is $\mathcal{H}_K = \mathcal{H}_x \otimes \mathcal{H}_y \otimes \mathcal{H}_z \otimes \mathcal{H}_p$ where the p-dimension is WHO. In PIC, Trust(p) *is* the WHO dimension — carried as sovereign state inside $C(p)$, anchored to p, accumulating through interactions. Without Trust anchoring, the six other sub-contexts are anonymous records. With Trust anchoring, they are participant-indexed sovereign state.

5.6 The Anchoring Topology

$$C(p) = \text{Trust}(p) \ \triangleright \ \{\text{Assets}(p), \text{Storage}(p), \text{Logic}(p), \text{Keys}(p), \text{Prefs}(p), \text{Intel}(p)\}$$

The symbol $\triangleright$ denotes structural anchoring — not hierarchical containment. Trust(p) does not contain the other six; it anchors them. Their participant-indexed character is derived from their entanglement with Trust(p).

5.7 The Entanglement Structure

Value in PIC is non-separable:

$$V(p) = \langle\, \text{Assets}(p) \otimes \text{Trust}(p) \,\rangle$$

Assets(p) without Trust(p) is not a PIC value object — it is anonymous content, a projection that destroys meaning. Trust(p) without Assets(p) is identity without value. The tensor product is the value. Neither factor alone is.

5.8 Context Inheritance and Shared Spaces

A participant's primary context represents their irreducible identity. Participants can also inherit additional contexts representing roles and shared application spaces.

Inherited role contexts (CEO, Trustee, Liquidity Provider) are additional contexts the participant inhabits, each with its own witness set, logic handlers, and interaction types. The participant's primary context remains free; personal operations proceed independently.

Shared inherited contexts (a DEX, a market, a governance space) are inhabited by multiple participants simultaneously. Each participant brings their sovereign state to the shared context. There is no global pool; the shared context provides shared computation — pricing rules, interaction handlers — but holds no shared value.

Theorem 5.4 (Native Interoperability)**.** *Cross-context value transfer requires no bridge protocol, no custodian, and no wrapped representation. The participant is the interoperability primitive.*

Source. By Property 4.3, only the participant can modify $C(p)$. Assets(p) is always located in the participant's context, never in a shared contract. Inherited contexts provide shared computation frames, not shared custody. The participant moves value between contexts by interacting within each — the interaction is the transfer mechanism, not a custody protocol. □

5.9 Context Height and Linearity

The height index h provides a causal record of state succession. $C(p)$ at height h is consumed to produce $C(p)$ at height $h+1$. Each height is unique: once assumed, it cannot be reused. For each participant and height, there exists at most one state. No branching; no forks; no reorganisation. This is structural — the witness set can only attest to one state per height, and contradicting this would require $2f+1$ witnesses to sign conflicting states.

5.10 Context as the Semantic Unit of Value

In information theory, the bit is the irreducible unit of information. In PIC, the Context Superstate $C(p)$ is the irreducible unit of value computation — the minimal object that PIC semantics operate on. Shannon showed that the bit's properties (copyability, observer-independence, combinability) determine the structure of information theory. PIC shows that the context's properties (linearity, sovereignty, participant-indexing) determine the structure of value theory.

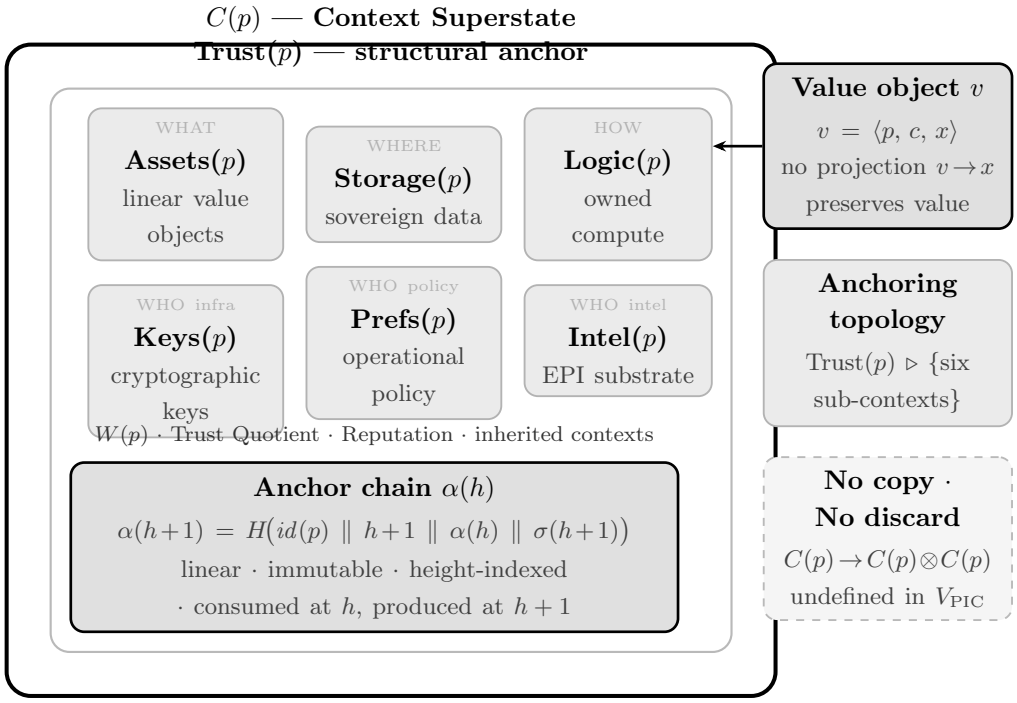

Figure 2: Context Superstate $C(p)$. Trust(p) structurally anchors the six sub-contexts. The anchor chain commits state succession at each height. Value object $v = \langle p, c, x\rangle$ is irreducible: no projection to x preserves value. The diagonal morphism is absent.

5.11 The Only Mechanism by Which Contexts Change

State transitions in the K-Machine occur only through interactions. There exists no transition of the form $\Sigma \to \Sigma'$. Any attempt to define global state evolution presupposes that value exists independently of participants, which contradicts the ontology established in Section 5.

Global ordering, global execution, and global reconciliation are not omitted by design. They are undefined by construction.

If a participant $p \notin P_\iota$, then $C(p)$ is neither read nor modified by interaction ι. Non-participants are not observers, validators, or witnesses. They are simply irrelevant. State transition is inseparable from participation. The act of interaction is the act of computation.

5.12 Formal Definition

Definition 5.5 (Interaction)**.** An interaction ι is a 5-tuple:

$$\iota = (S,\, T,\, W,\, f_\iota,\, \mathrm{Auth}_\iota)$$

where:

- $S \subseteq P$ is the source participant set (contexts consumed),
- $T \subseteq P$ is the target participant set (contexts produced),
- W is the witness set — $W(p)$ for each $p \in S \cup T$, embedded in $\mathrm{Trust}(p)$,
- f_ι is the linear transformation applied over $S \cup T$, and
- $\Sigma = \{\mathrm{sig}(p) : p \in S\}$ is the cryptographic authorisation from all source participants.

The abstract form $\iota = (P_\iota, C_\iota, W_\iota, f_\iota)$ is equivalent, with $P_\iota = S \cup T$.

The transformation τ induces:

$$\tau : \bigotimes_{p \in S \cup T} C(p) \longrightarrow \bigotimes_{p \in S \cup T} C'(p)$$

This transformation consumes exactly one valid joint state, produces exactly one new joint state, and admits no intermediate state. Linearity is structural.

5.13 Four Structural Properties

(1) Atomicity. Execution is defined if and only if $\forall p \in S \cup T,\ p$ participates. If any constituent is absent, the transition does not exist. There is no rollback because there is no partial state to roll back.

(2) Linearity. Each $C(p) \in C_\iota$ is consumed exactly once. There is no copying of $C(p)$, no discarding of intermediate state, and no duplication of value. This is structural — no locks, no rollback mechanisms, no external arbitration.

(3) Locality (contextual scope). The transition f_ι operates exclusively over $\{C(p) \mid p \in S \cup T\}$ and their witness set W_ι. No context outside this set is read, modified, or observed.

Locality in PIC is not geographic or topological. It is logical and witness-bounded: the transformation is local to the named participant contexts and their linear witnesses — the complete and natural scope of the computation. A participant in Tokyo and a participant in New York, both named in the same interaction, are local to each other in this sense. Locality is defined by WHO is in the transformation, not WHERE they are. The ICS is not a restriction of a larger scope; it is the total scope.

(4) Determinism. Given $S \cup T$, their current contexts, and the interaction specification, f_ι is deterministic. All information required to determine the outcome is locally available.

5.14 Consent as a Semantic Primitive

Property 5.6 (Consent)**.** An interaction ι requires the explicit cryptographic authorisation of all source participants: $\mathrm{Auth}_\iota = \{\mathrm{sig}(p) : p \in S\}$ is a required element of ι. Without it, f_ι is undefined.

Consent is not an authorisation check performed against a ledger. Consent *is* participation. A participant who does not participate cannot be affected. There exists no mechanism by which value can be transferred from a participant without their participation. This is not a security guarantee. It is a semantic constraint.

5.15 The Interaction Context Space

Definition 5.7 (Interaction Context Space)**.** The ICS for interaction ι is:

$$\mathrm{ICS}(\iota) = (O,\ P_\iota,\ C_\iota)$$

where O is the operator, $P_\iota = S \cup T$ is the participant set, and $C_\iota = \bigoplus_{p \in P_\iota} C(p)$ is the composite context formed by all named participants. The witness set $W(\iota)$ satisfying the threshold requirement completes the ICS. How witnesses are selected and time-slotted is determined by the implementing protocol.

The ICS is the complete and natural scope of the interaction — defined by the participants themselves. It is not a partition of the network, not a shard, not a restricted committee. An interaction involving two participants has a two-participant ICS. An interaction involving a thousand participants has a thousand-participant ICS. The protocol imposes no ceiling.

5.16 Interactions Are Not a Sequence

In Contextual Compute, interactions are not updates to a timeline. They are *transformations of entanglement*. Each interaction is scoped to its participants and witnesses; its effects are contextual and complete. Interactions do not interleave: they either share participants (in which case linearity enforces exclusivity) or they do not (in which case ordering is irrelevant). There is no global clock, no canonical history, no total order.

Time is an external observer construct. In value systems, observers cannot be factored out; time must therefore be local to context. Attempting to impose a global time reintroduces an external frame and collapses the model back into Cartesian structure.

5.17 Finality

Finality in PIC is intrinsic. An interaction either completes or it does not; there is no reordering, rollback, or probabilistic convergence. Formally, if:

$$C_\iota \xrightarrow{f_\iota} C'_\iota$$

then C'_ι is final by construction.

Finality is the completion of a linear transition. Once an interaction executes, its result is final for all participants involved:

$$C(p) \mapsto C'(p) \ \text{ is final for participant } p$$

This is analogous to collapse in quantum systems: once a joint state resolves, there is no prior state to return to.

Finality is contextual because value is contextual. Global finality is undefined because global value is undefined.

In Cartesian systems, ordering precedes finality. In Contextual Compute, finality is intrinsic and ordering never arises. Finality is not the result of time passing. It is the result of interaction completion.

5.18 What Interaction Excludes

An interaction in PIC cannot: read anonymous global state (Theorem 5.1, which establishes that state is always participant-indexed); produce externally visible intermediate state (f_ι is atomic); re-enter a context already consumed (linearity — the pre-context does not persist through execution); or impose ordering on concurrent interactions over disjoint participant sets (Axiom 6.6, below).

5.19 The Complexity Consequence

Systems maintaining global replicated state incur cost $O(N \times T \times C)$ per transition, where N is network size. For PIC, the cost per interaction is $O(k \times t)$, where $k = |S \cup T|$ is the number of named participants and t is interaction complexity. N does not appear in the cost function. This is not a constant-factor improvement. It is a complexity class separation: N has been removed entirely.

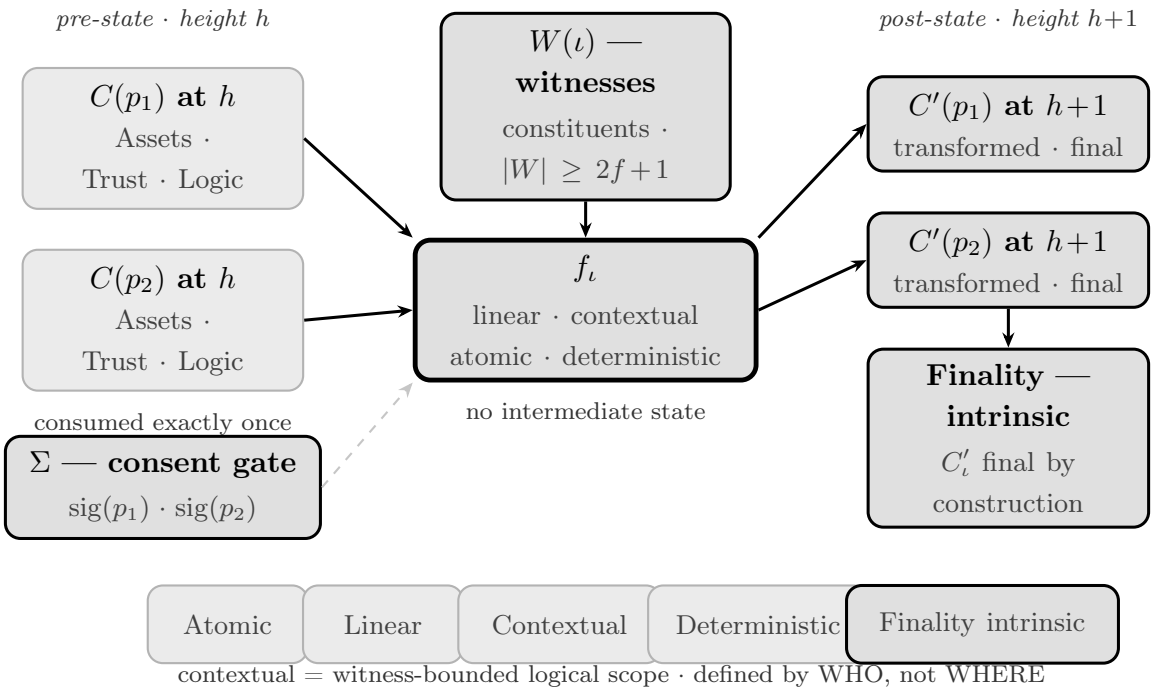

Figure 3: Interaction $\iota = (S, T, W, f_\iota, \mathrm{Auth}_\iota)$ as a linear transformation. f_ι is gated by consent Auth_ι and attested by witnesses $W(\iota)$. Pre-contexts consumed exactly once; post-contexts final. "Contextual" means witness-bounded logical scope, not geographic proximity.

5.20 Agreement Is Semantic, Not Procedural

PIC does not eliminate communication or message passing. What PIC eliminates is *external agreement over replicated state*: the requirement that nodes holding copies of global state must run a protocol to determine which copy is correct.

Agreement in PIC is semantic, not procedural. Because all constituents of the value function — participants and witnesses — are known a priori, the interaction defines its own agreement scope. There is nothing outside the interaction that must agree. This is not consensus without ordering. It is agreement without replication. The distinction is categorical.

5.21 The Categorical Distinction

A *validator* is an external observer. A *witness* is a constituent.

Validators observe replicated state and attempt to agree on its correctness; agreement is discovered after execution. Witnesses participate in the value interaction itself; their participation *constitutes* agreement. Execution and agreement are the same act.

Adding or removing a witness changes the interaction itself. A different witness set implies a different value — not a different confirmation of the same value, but a different value object. Witnesses are not metadata. They are part of the entanglement.

5.22 Witnesses Are Statefully Embedded in Trust

Witnesses are not assigned to interactions at runtime. They are statefully embedded in Trust(p) — the t-dimension of the Context Superstate. $W(p)$ is part of $C(p)$, committed in the anchor chain, carried by the participant across all interactions.

A validator set is external to the state it validates. A witness set in PIC is internal to the participant's state — it is part of WHO the participant is. Changing the witness set changes the participant's trust structure, which changes the value objects they can form.

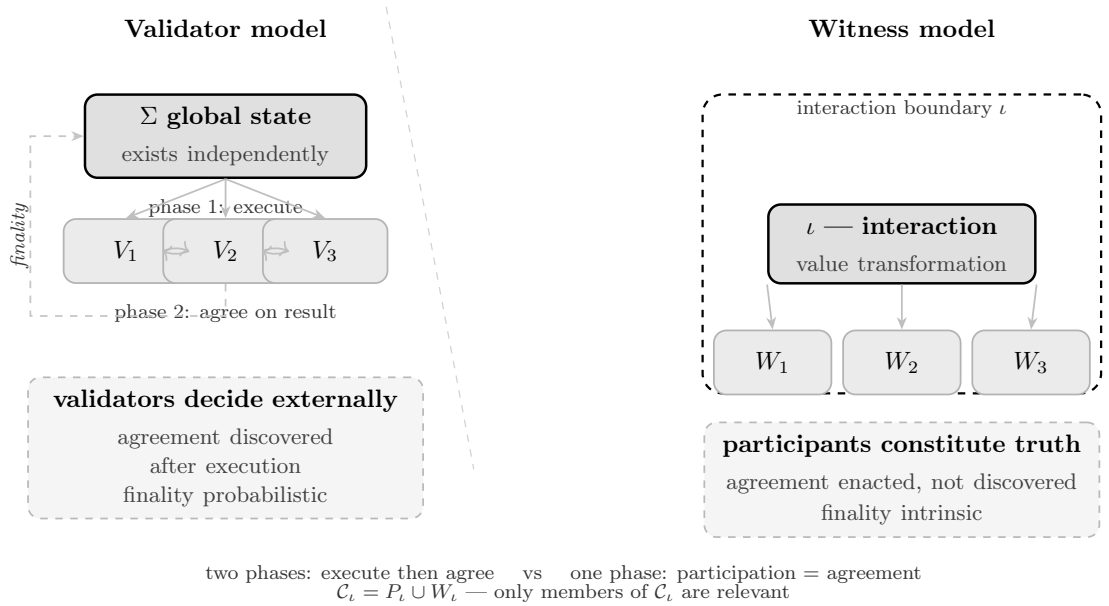

Figure 4: Validator model (left) vs witness model (right). Validators are external observers who agree on replicated state in a second phase. Witnesses are constituents inside the interaction boundary: their participation constitutes agreement. $\mathcal{C}_\iota = P_\iota \cup W_\iota$ is the total scope of the interaction.

5.23 The Full Constituent Set

Definition 5.8 (Constituent Set)**.** For interaction ι with participant set P_ι and witness set W_ι, the full constituent set is:

$$\mathcal{C}_\iota = P_\iota \cup W_\iota$$

Only members of $\mathcal{C}_\iota$ are relevant to the interaction.

Definition 5.9 (Value over Joint Constituent State)**.** Value is defined over the joint state of all constituents:

$$V_\iota = \bigotimes_{c \in \mathcal{C}_\iota} C(c)$$

There exists no notion of a witness-independent value.

5.24 Witness Set Formal Specification

Definition 5.10 (Witness Set)**.** The witness set $W(\iota)$ for interaction ι satisfies $|W(\iota)| = 3f+1$ where f is the maximum number of Byzantine witnesses tolerated. Validity requires:

$$\text{Validity}(\iota) \iff \big|\{w \in W(\iota) : \text{Att}(w) \text{ valid}\}\big| \geq 2f+1$$

The composition of $W(\iota)$ — how witnesses are selected, weighted, or distributed — is determined by the implementing protocol. CC Protocol specifies the threshold requirement. Any mechanism satisfying $2f+1$ of $3f+1$ is compliant.

Witnesses derive authority from participants, not from their own weight. A witness's role exists only in relation to the participant context it is anchored to. A witness not anchored to a participant is not part of the value function and carries no authority. Adding witnesses to an interaction increases the strength of attestation — the assurance that the transformation occurred correctly — but does not transfer control. The participant whose context is being transformed retains sovereignty throughout. This differs fundamentally from validator sets in classical systems, where validators derive authority from their position in an external agreement process, independent of any participant.

5.25 Witnesses Across Inherited Contexts

When participant p inhabits an inherited context, the witness set for operations within that context is $W(\text{inherited context})$, not $W(p)$. Different contexts carry different security parameters without fragmenting the participant's identity or requiring custody transfer. The participant's primary context remains free.

5.26 The Value–Agreement Equivalence

Theorem 5.11 (Value = Agreement)**.** *For any interaction ι, value exists if and only if all constituents of ι participate.*

Proof Sketch. Value is defined as a linear transformation over the joint state $V_\iota = \bigotimes_{c \in \mathcal{C}_\iota} C(c)$. If any constituent $c \in \mathcal{C}_\iota$ does not participate, the transformation is undefined — the joint state is incomplete and τ cannot execute. If all constituents participate, τ executes, the joint state transforms, and value resolves. There exists no intermediate or external criterion. □

This theorem replaces consensus. Consensus presupposes replicated state and asks which replica is correct. Theorem 5.11 shows there are no replicas — value is constituted by participation, not discovered by comparing copies.

5.27 Derived Corollaries

Corollary 5.12 (Double-Spend Undefined)**.** *$C(p)$ at height h exists once. An interaction consuming it produces $C(p)$ at height $h+1$, destroying the pre-condition any second interaction would require. The operation is not rejected — it is absent from the grammar.*

Corollary 5.13 (Social Value)**.** *Value exists in witness agreement, not in content alone. An identical asset with no witnesses has no value. The witness set is constitutive of value, not confirmatory.*

Corollary 5.14 (Contextual Value)**.** *The same asset in different inherited contexts has different value — because the witness set and interaction frame differ. Value is participant–context–witness entangled.*

Corollary 5.15 (No Orphaned Value)**.** *Every value object has an owning participant p, a context c, and a witness set $W(p)$. Value without a participant — an anonymous balance in a contract — is not a PIC value object. It is information that references value.*

5.28 Why Witnesses Replace Consensus: The Cartesian Recursion

5.28.1 The Recursion

In any Cartesian system — one where state is copyable and exists independently of who holds it — the following recursion governs every attempt to determine the value of state:

To know the current value of σ, one needs agreement on which σ is current. To reach agreement on σ, one needs to know which transitions are valid. To know which transitions are valid, one needs to know the current value of σ.

The circle never closes. It is not a slow circle; it has no base case. More precisely: *a definitive state is needed to determine valid transitions, but valid transitions are needed to define the definitive state.* This recursion is not accidental. It is structural.

In Cartesian systems, state is copyable; therefore multiple representations of value necessarily exist. Once multiple representations exist, agreement must be externalised. Consensus becomes the mechanism that enforces linearity on a substrate that does not admit it. But consensus itself requires a reference state — there is no definitive state until consensus completes, and consensus cannot complete until a definitive state exists. This is permanent recursion. No increase in speed, stake, or cryptographic sophistication can remove it.

Formally: consensus requires a global predicate $\text{Agree}(\Sigma_{\text{global}})$. But Σ_{global} is structurally unavailable in PIC (Theorem 5.1): state is always participant-indexed, and $\Sigma = \{C(p)\}_{p \in P}$ is already complete and authoritative without consensus. The predicate has no referent. The question consensus was designed to answer becomes irrelevant.

5.28.2 The PIC Resolution: Participant as Definitive Anchor

In PIC the recursion does not arise. The base case exists: the participant. $C(p)$ at height h exists in exactly one location — anchored to $\text{Trust}(p)$, committed in $\alpha(h)$, witnessed by $W(p)$. It is not a copy among equals. It is *the* state, by construction.

Cartesian	**PIC**
σ can be anywhere	$C(p)$ exists at one height, one anchor
All copies equally valid	The participant is the definitive anchor
No base case	Recursion never arises
Agreement requires agreement	Agreement is constituted, not computed

5.28.3 Why Consensus Cannot Be Added Back

One might attempt to layer consensus atop PIC. This is impossible without breaking the model. Adding consensus would require copying contexts, externalising ownership, and reintroducing global state. Each step violates linearity and participant sovereignty. Consensus is not an optional module. It belongs to a different category.

5.28.4 The Structural Statement

The Domain Separation Theorem establishes that the diagonal morphism enabling copying is a Cartesian structure with no counterpart in the linear category. Removing it does not merely make copying harder. It makes the question "which copy is current?" *structurally non-expressible*. The recursion that requires consensus does not become hard to close. It becomes impossible to state.

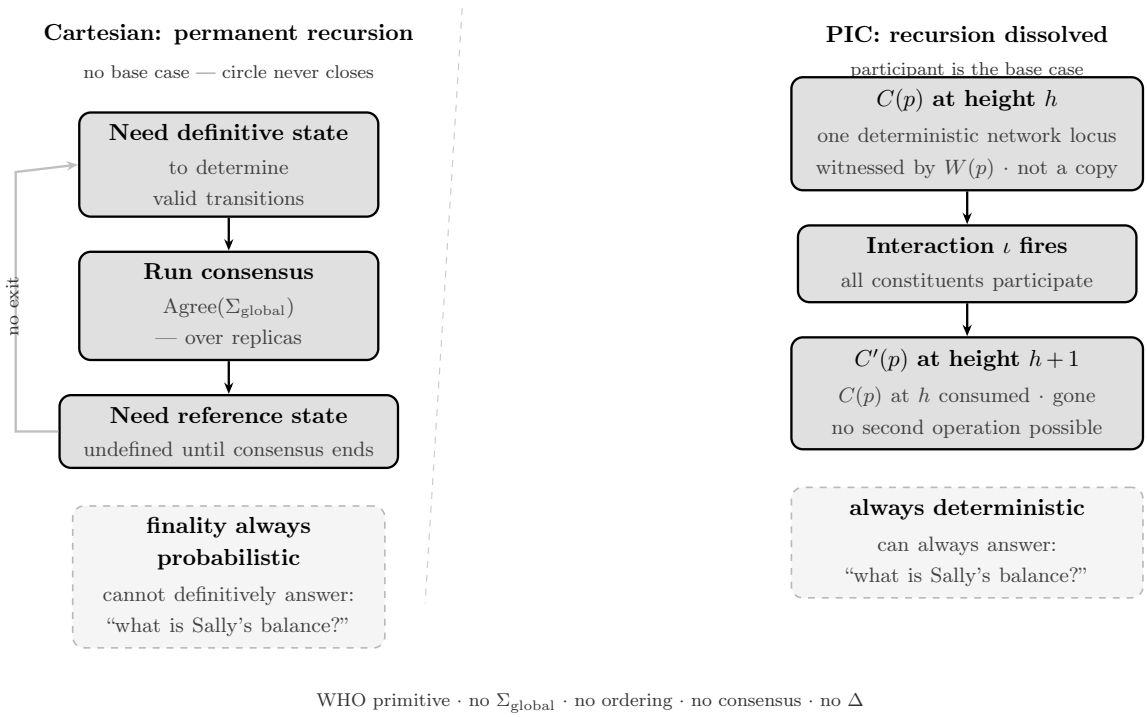

Figure 5: The Cartesian consensus recursion (left) has no base case. In PIC (right), $C(p)$ at height h is the definitive anchor: one deterministic network locus, witnessed by $W(p)$, not a copy. The question "which copy?" cannot be stated in PIC; "what is Sally's balance?" always has a deterministic answer.

In classical systems, fault tolerance is achieved by replication: multiple copies of state are maintained so that Byzantine nodes cannot corrupt the single authoritative copy. Replication requires Cartesian structure; fault tolerance therefore appears to require Cartesian state.

PIC demonstrates that this coupling is not necessary. Fault tolerance is achieved through the witness threshold — $|W(p)| = 3f + 1$, requiring $2f + 1$ attestations for validity. This is identical to classical BFT. But it operates over a linear state space, not a replicated one.

Classical BFT	**PIC**
Fault tolerance via replication	Fault tolerance via witness threshold
Multiple copies → agree on one	One state → witnesses attest
Linearity and FT coupled	Linearity and FT independent layers

No forks. No reorgs. Forks arise when multiple copies of state diverge. In Contextual Compute there are no copies, no shared history, and no global order. Each interaction produces a single, irreversible transformation. Forks are undefined.

5.29 Closing the Loop

The witnessing model closes the semantic loop opened by the Domain Separation Theorem. The companion paper establishes that value and information are categorically distinct. PIC's witnessing model shows what computation over value actually looks like: participant-indexed, linear, witnessed, constitutively final.

This is not an optimisation of consensus. It is a categorical shift — from discovering agreement over replicated copies to constituting agreement through participant witnessing.

6. Network Objects: The First Linear Primitive at Network Level

6.1 The Semantic Gap

PIC defines the semantics of participant-indexed linear value computation. Between the model and execution on a classical substrate lies a gap: classical networks are built on Cartesian primitives. Objects are copyable; state is replicable; there is no native representation of a linear type at the network level.

This gap cannot be closed by enforcement. Adding locks, ordering constraints, and consensus protocols to a Cartesian network simulates linear behaviour without achieving it. The Cartesian diagonal morphism remains in the substrate — suppressed by protocol, not eliminated by structure.

The gap requires a new primitive: a first-class linear type at the network level. This is the *Network Object*.

6.2 The Distinction

Local Object (Cartesian): "What is Sally's balance?" The balance is content in anonymous global state. It can be read, copied, and replicated freely. Multiple nodes hold equally valid copies. The question has no definitive answer without consensus. Sally is not present in the question.

Network Object (Linear): "What is Sally's agreed balance?" The balance is a non-separable joint state of $\text{Assets}(\text{Sally}) \otimes \text{Trust}(\text{Sally})$. It exists at one height, in one Context Superstate, anchored by one $\alpha(h)$, witnessed by $W(\text{Sally})$. The question has a definitive answer without consensus. Sally is constitutive of the answer.

Definition 6.1 (Network Object)**.** A Network Object is a computational element that is:

- **Stateful** — maintains persistent participant-indexed state,
- **Linear** — cannot be copied; linearity is structural, not enforced,
- **Participant-bound** — entangled with a specific WHO,

- **Witness-embedded** — carries its agreement proof within the object,
- **Network-level** — exists at the participant-context locus: participant-indexed, witness-anchored, context-scoped. Whether the network has one node or one million does not change the object's nature. Each participant context and each interaction carries independent characteristics — trust requirements, witness sets, logic, policy — that a single global ordering queue cannot represent regardless of its speed. The participant-context locus eliminates the single ordering line not to increase throughput but because the line cannot express the semantic diversity of participant contexts, and
- **Deterministic** — transitions produce identical outputs given identical inputs.

A Context Superstate $C(p)$ is a Network Object.

6.3 The Significance

Network Objects provide a primitive for linear types at the network level — a computational element whose linearity is structural rather than enforced. All prior distributed systems primitives are Cartesian:

Primitive	**Properties**	**Domain**
TCP segment	Copyable, replicable	Information
HTTP resource	Copyable, addressable	Resource
Database row	Copyable, queryable	Data
Network Object	Linear, participant-bound, witness-embedded	Value

Linearity is not a property the application enforces on top of a Cartesian substrate. It is a property of the primitive itself.

6.4 Scope

The semantic definition of Network Objects belongs in PIC; the full specification — the ISM execution environment and its implementation primitives — is the subject of the subsequent implementation paper (CC5, Sarva Labs, forthcoming).

7. The Six Axioms and PIC as Category

7.1 Axioms as Consequences

The axioms of PIC are not postulates adopted for mathematical convenience. They are necessary consequences of the K-Machine's structure — each forced by a specific property established in the preceding sections. An axiom that could be removed without violating the K-Machine's requirements would be a design choice, not an axiom. None of the six can be removed.

7.2 The Six Axioms

Axiom 7.1 (Conservation)**.** In any interaction ι, total value across all participating contexts is preserved:

$$\sum_{p_i \in S \cup T} V\big(C'(p_i)\big) \;=\; \sum_{p_i \in S \cup T} V\big(C(p_i)\big)$$

Source. The K-Machine's transition function is a linear transformation. Linear transformations in a symmetric monoidal category preserve the tensor product — they do not introduce or eliminate objects.

Axiom 7.2 (Consent)**.** A participant's context changes if and only if the participant consents. No external process can modify $C(p)$ without the participant's explicit cryptographic authorisation.

Source. Property 4.3. Modifications to σ can only originate from within the participant's own interaction. An interaction without source authorisation $\Sigma = \{\text{sig}(p) : p \in S\}$ is not an unauthorised interaction — it is undefined within the PIC model.

Axiom 7.3 (Linearity)**.** Each Context Superstate $C(p)$ is consumed exactly once upon transition to $C'(p)$. No context can be duplicated, forked, or partially consumed. The following operations are undefined:

$$C(p) \to C(p) \otimes C(p) \quad \text{(copy)} \qquad C(p) \to \mathbf{1} \quad \text{(discard)} \qquad C(p) \to C(p_1) \otimes C(p_2) \quad \text{(fork)}$$

Source. V_{PIC} is symmetric monoidal but not Cartesian. The diagonal morphism $\Delta : A \to A \otimes A$ does not exist. These are categorical results, not security results achieved by enforcement.

Axiom 7.4 (Witness Sufficiency)**.** An interaction is valid if and only if:

$$\text{Validity}(\iota) \iff \big|\{w \in W(\iota) : \text{Att}(w) \text{ valid}\}\big| \geq 2f + 1$$

Source. Theorem 5.11 (Value = Agreement). The locus of agreement for any interaction is the ICS — defined by the named participants, independent of network size N. Agreement scoped to the ICS is not restricted agreement. It is complete agreement over the complete state the interaction concerns.

This is categorically distinct from partitioning strategies applied to global Cartesian state. Sharding and layered architectures divide a global state too large to order. PIC has no global state to partition. Byzantine failures within one ICS do not affect participants whose ICS is disjoint. This is not an optimisation of consensus. It is a categorical shift — from discovering agreement over replicated copies to constituting agreement through participant witnessing.

Axiom 7.5 (Composability)**.** For interactions I_1 and I_2 where $\text{output}(I_1) = \text{input}(I_2)$, their composition $I_2 \circ I_1$ is a valid interaction in V_{PIC}. Composition preserves all prior axioms.

Source. Categorical structure requires that morphism composition is well-defined. Composition depth does not increase per-interaction cost — each interaction costs $O(k \times t)$ for its own participants, independently.

Axiom 7.6 (Commutativity)**.** For interactions I_1 over P_1 and I_2 over P_2 with $P_1 \cap P_2 = \emptyset$:

$$I_1 \circ I_2 = I_2 \circ I_1$$

Source. Axiom 7.3 (Linearity) and Locality (Section 5.13). When $P_1 \cap P_2 = \emptyset$, there is no context that both I_1 and I_2 touch.

7.2.1 The Causal Record

Within a participant's own context, the anchor chain records the causal succession of states. This is a structural causal record — not a serialisation mechanism. $C(p)$ at height h is consumed by whichever interaction executes; $C(p)$ at height $h + 1$ is produced; $C(p)$ at height h no longer exists. There is nothing to serialise.

7.2.2 The Partial Order Objection

Partial ordering still presupposes shared state. Partial orders exist to reason about causality among replicated effects. In Contextual Compute, causality is participant-scoped. If two interactions do not share participants, there is no causal relationship to model. If they do share participants, linearity already enforces a single valid transformation. Ordering adds nothing.

7.2.3 Commutativity Within Shared Inherited Contexts

The critical distinction is not shared space — it is shared state.

In classical computation, state is anonymous and globally indexed. A shared computational space necessarily implies shared state: participants in the same context are reading and writing the same global object. Ordering is therefore required whenever a shared space exists, because the state itself is shared. This is why partial commutativity approaches — which demonstrate that operations over *disjoint owned objects* commute — collapse precisely at shared objects. Any operation touching globally shared mutable state requires total ordering. Partial commutativity holds where applications are simple; it fails where they are interesting.

PIC makes a categorically different claim. State is participant-indexed. A shared computational context does not imply shared state, because state is indexed to the participant for that context — not to the context itself.

Formally: in a shared inherited context C_{shared} inhabited by participants $p_1, \ldots, p_n$, each participant brings their own context:

$$C(p_i, C_{\text{shared}}) \quad \text{for each } p_i$$

These are distinct objects. $C(p_i, C_{\text{shared}}) \neq C(p_j, C_{\text{shared}})$ for $i \neq j$. The shared context contributes:

- $\text{Logic}(C_{\text{shared}})$ — shared computation: pricing rules, execution handlers, interaction types
- $\text{Prefs}(C_{\text{shared}})$ — shared parameters: operational policy

It does not contribute shared value state. $\text{Assets}(p_i)$ remains in $C(p_i, C_{\text{shared}})$ for each participant. There is no shared pool. There is no shared object for operations to contend over. Commutativity therefore holds structurally — not by proof of independence for specific operation subclasses, but because the condition that creates dependence is absent.

Writing to the shared context itself — modifying $\text{Logic}(C_{\text{shared}})$ or $\text{Prefs}(C_{\text{shared}})$ — is a governance action scoped to the owner of that context. It is rare, deliberate, and explicitly authorised. It is categorically distinct from the routine participant interactions that occur within the shared space.

Two levels of ordering.

- **Protocol level:** order-free by construction. Participant contexts are always distinct, even inside shared computational spaces. No protocol mechanism imposes ordering over participant interactions.
- **Application level:** ordering is available if the domain requires it. A market enforcing price-discovery sequence or order-book priority implements that ordering in $\text{Logic}(C_{\text{shared}})$. It is an application choice — not a protocol constraint. The protocol imposes nothing. The application imposes exactly what its business rules require and nothing more.

Contrast with weakly commutative Cartesian systems. Weakly commutative systems achieve partial commutativity by proving that specific operation subclasses over disjoint owned objects produce identical results regardless of order. This is correct and useful within the Cartesian model. It does not change the substrate: state remains anonymous and globally indexed. Commutativity is a proved property of certain operation pairs. It is not a structural property of the state space. The moment a shared object is touched, the proof fails and ordering is imposed.

PIC commutativity is structural. It holds universally across all participant context pairs, including within shared inherited spaces, because the substrate itself does not admit shared value state. There are no operation pairs requiring independence proofs. There is nothing to prove independence of.

7.3 Derived Properties

1. **State completeness without aggregation.** From Axioms 7.2 and 7.3. The family $\Sigma = \{C(p)\}_{p \in P}$ is the complete and authoritative network state. Anonymous global aggregation adds nothing and requires what participant-indexed state structurally prevents.
2. **ICS-scoped agreement.** From Axiom 7.4. Network size N plays no role; agreement at the ICS is complete, not approximate.
3. **Structural parallelism.** From Axioms 7.3 and 7.6. Parallelism does not collapse at shared application spaces.
4. **Complexity class separation.** $O(N \times T \times C)$ vs. $O(k \times t)$. N removed from the cost function entirely.

7.4 PIC as a Category

Definition 7.7 (PIC Value Category V_{PIC})**.** V_{PIC} is defined as follows:

- **Objects:** participant-indexed context pairs $(p, C(p))$.
- **Morphisms:** interactions $I : (p_1, C(p_1)) \times \cdots \times (p_n, C(p_n)) \to (p_1, C'(p_1)) \times \cdots \times (p_n, C'(p_n))$.
- **Identity:** $\text{Id}_{(p,C(p))}$ — the null interaction.
- **Composition:** $I_2 \circ I_1$ where output contexts of I_1 are input contexts of I_2, preserving all six axioms.
- **Monoidal structure:** the tensor product $\otimes$ over participant contexts; V_{PIC} is symmetric monoidal.
- **Commutativity:** for morphisms over disjoint objects, $I_1 \otimes I_2 = I_2 \otimes I_1$.
- **Critical structural absence:** no diagonal morphism $\Delta : (p, C(p)) \to (p, C(p)) \otimes (p, C(p))$ exists; no counit morphism $\varepsilon : (p, C(p)) \to \mathbf{I}$ exists. V_{PIC} is symmetric monoidal but not Cartesian.

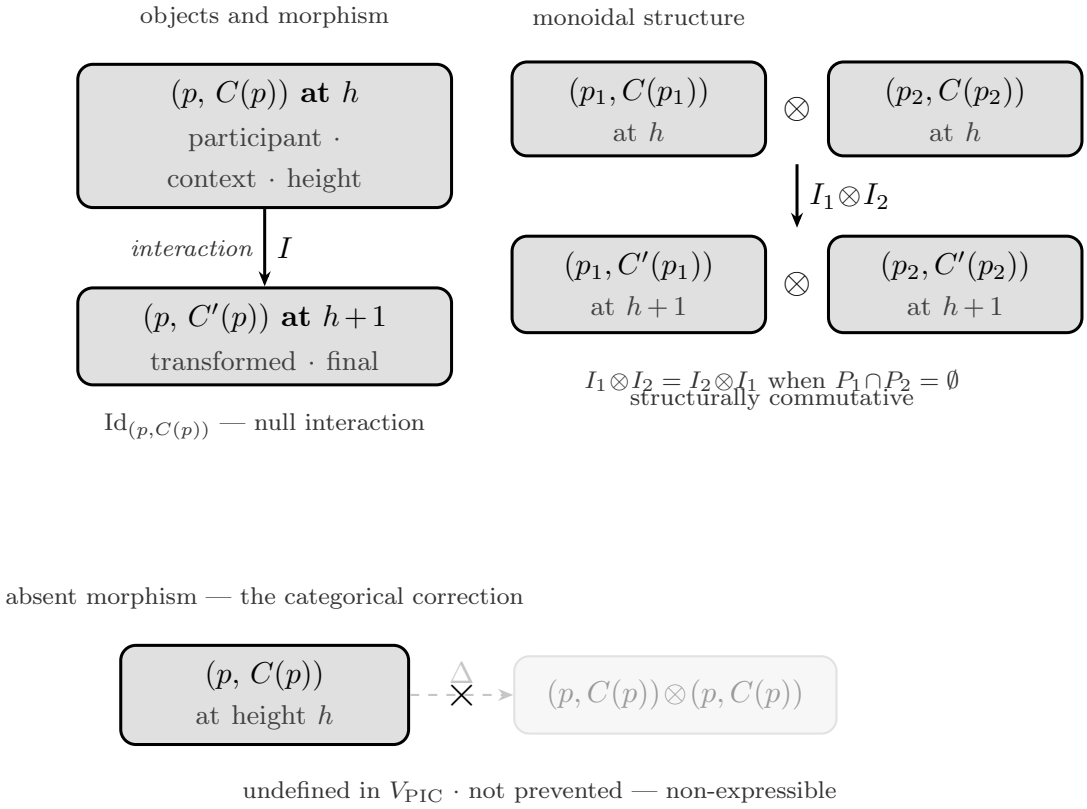

Figure 6: The PIC value category V_{PIC}. Objects: participant-indexed context pairs. Morphisms: interactions. The monoidal structure $\otimes$ is symmetric: interactions over disjoint participant sets commute structurally. The diagonal morphism $\Delta : C(p) \to C(p) \otimes C(p)$ is absent — copying is not prevented but non-expressible.

$$\mathbf{C}_{\text{Cartesian}} \subsetneq V_{\text{PIC}}$$

Every classical computation is a morphism in V_{PIC} at WHO $= 0$. The inclusion is strict: V_{PIC} contains morphisms operating over participant-indexed linear state spaces with no counterpart in $\mathbf{C}_{\text{Cartesian}}$.

Dimension	**Classical Models**	**Contextual Compute**
Ontology	Information	Value
Category	Cartesian monoidal	Symmetric monoidal (linear)
State space	Anonymous	Participant-indexed
Copying	Native	Undefined
Agreement	External consensus	Constitutive participation
Ordering	Required	Irrelevant
Finality	Probabilistic, global	Deterministic, contextual
Observers	External validators	Internal witnesses
Failure mode	Forks, reorgs	Undefined (cannot occur)

Optimisations such as faster consensus, reduced ordering, object independence, and parallel execution remain within the Cartesian paradigm. They attempt to minimise *when* ordering is required but cannot eliminate *why* ordering exists. Contextual Compute eliminates the need for ordering by operating in a domain where ordering has no meaning.

7.5 What PIC Eliminates

Mechanism	**Status in V_{PIC}**	**Categorical reason**
Global consensus	Undefined	No replicated copies to agree on
Total ordering	Undefined at protocol level	No shared value state to sequence
Shared object ordering	Undefined	Inherited contexts carry no shared value
Double-spend	Non-expressible	No diagonal morphism — $C(p)$ cannot be copied
Ordering-dependent extraction	Undefined	No shared ordering resource
Re-entrancy	Non-expressible	Linear consumption — pre-state does not persist
Global atomicity	Undefined	No anonymous global state; scope bounded to named participants
Anonymous global state	Structurally unavailable	State is always participant-indexed; $\Sigma = \{C(p)\}_{p \in P}$ is already complete

The right column says "Undefined" not "Impossible." Impossible means the operation can be expressed, attempted, detected, and rejected. Undefined means the operation cannot be expressed within the grammar of V_{PIC}. There is nothing to detect. There is nothing to reject. The morphism does not exist in the algebra.

8. The Derivation Chain and Bridge to Implementation

8.1 The Architecture as Categorical Entailment

Every element of PIC follows as a categorical consequence of the K-Machine's structure. The derivation chain makes this explicit:

K-Machine: WHO is a fundamental dimension; state must be participant-indexed.

$\Rightarrow$ **Participant semantics** (§4). $p = (id, \sigma, f_p, H)$ — irreducible, sovereign, persistent. PIC triple (P, I, C) — three irreducible semantic primitives.

$\Rightarrow$ **Context Superstate semantics** (§5). $C(p) = \mathrm{Trust}(p) \triangleright \{\mathrm{Assets}, \ldots, \mathrm{Intel}\}$. Value object $v = \langle p, c, x \rangle$ — no projection preserving value. Participant-Indexed State Completeness — $\Sigma = \{C(p)\}_{p \in P}$ is complete.

$\Rightarrow$ **Interaction semantics** (§5.10). $\iota = (S, T, W, f_\iota, \mathrm{Auth}_\iota)$ — the only mechanism by which contexts change. Atomic, linear, contextual, deterministic, consensual. Finality intrinsic.

$\Rightarrow$ **Witness semantics** (§5.19). $\mathcal{C}_\iota = P_\iota \cup W_\iota$; $V_\iota = \bigotimes_{c \in \mathcal{C}_\iota} C(c)$. Value = Agreement (Theorem 5.11). Cartesian recursion dissolved.

$\Rightarrow$ **Axioms** (§7). Conservation, Consent, Linearity, Witness Sufficiency, Composability, Commutativity. V_{PIC}: symmetric monoidal, not Cartesian.

Each row is forced by the row above. The architecture is not assembled from components. It is the entailment of a single premise: the participant is a computational primitive.

1. **Sovereign state.** $C(p)$ is participant-owned, anchored, and modifiable only by the participant's consent.
2. **Structural parallelism.** Full commutativity across independent participant contexts — including within shared inherited contexts.
3. **Intrinsic finality.** Constituted at the moment of witness attestation. The pre-context is consumed. The transition is complete.
4. **Complexity independence from network scale.** $O(k \times t)$ per interaction; N absent from the cost function.
5. **Native interoperability.** The participant is the interoperability primitive; no bridge protocol required.

8.2 What PIC Does Not Specify

PIC is an operational semantics. It specifies what the computation means, not how it is executed on a physical substrate. Three questions are deliberately left open: how interactions are physically transmitted (wire format — the Context Protocol and IFP); how witness agreement is reached in practice (determined by the implementing protocol using any mechanism satisfying the $2f + 1$ threshold); and how participant state is stored and retrieved (the Context Protocol participant chain specification).

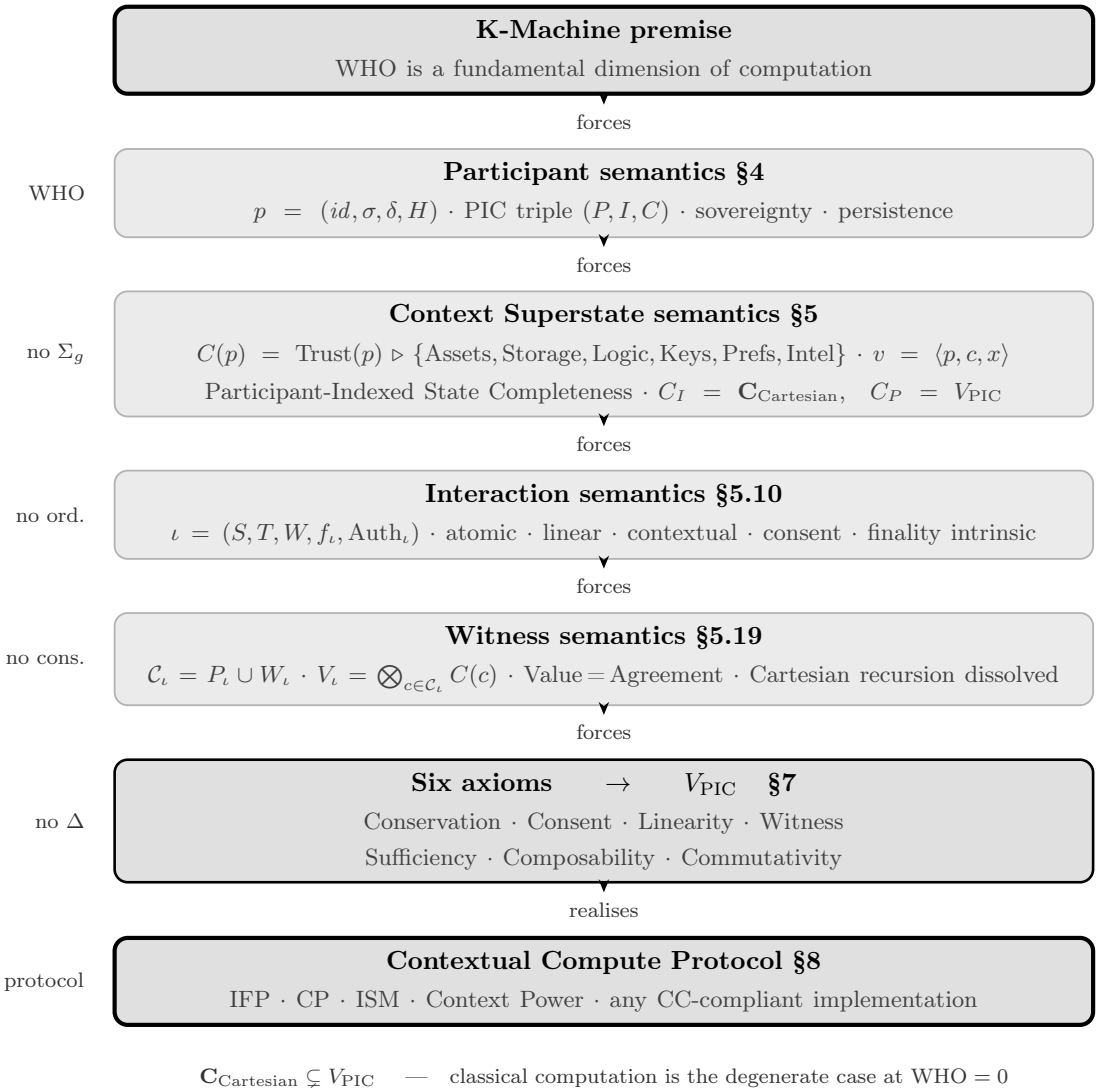

Figure 7: The derivation chain. Every level is forced by the one above. The architecture is not assembled from design choices — it is the entailment of a single premise: the participant is a computational primitive. $\mathbf{C}_{\mathrm{Cartesian}} \subsetneq V_{\mathrm{PIC}}$: classical computation is the degenerate case at WHO = 0.

This boundary is not accidental. Separating semantics from implementation allows the categorical properties of V_{PIC} to be evaluated independently of any specific implementation choice.

8.3 The Bridge to Implementation

PIC semantic object	**CC Protocol realisation**
Participant $p = (id, \sigma, f_p, H)$	Network-addressable entity with sovereign key material
Context Superstate $C(p)$	Context Protocol — succession, consent, participant state specification
Interaction $\iota = (S, T, W, f_\iota, \mathrm{Auth}_\iota)$	ISM interaction type — typed domain instruction over named participant contexts
Witness set $W(p)$	$\lvert W(\iota)\rvert = 3f + 1$, threshold $2f + 1$ — composition determined by implementing protocol
ICS formation	Composite context $C_\iota = \bigoplus C(p)$ across named participants; witness set satisfying threshold; fully determined before execution begins
Value agreement $V = \mathrm{Agreement}$	ISM execution — typed domain primitives over linear state
Anchor chain $\alpha(h)$	Context Protocol succession edges — vertical height succession within each participant's context chain

Each row is a direct realisation. There is no translation layer, no impedance mismatch, no approximation. The semantic objects were defined precisely so that their implementation could be direct.

8.4 The Remaining Frontier

Three extensions remain beyond the scope of this paper:

Post-quantum security. Keys(p) supports both classical and post-quantum signature schemes with interaction-scoped cryptographic strength. The full specification of the quantum-resistant witness protocol is addressed in the security extension.

Zero-knowledge composition. PIC's linear state space composes naturally with zero-knowledge proof systems. The formal treatment of ZK-PIC is addressed in the privacy extension.

Agent delegation. PIC's consent model and inherited context structure provide the semantic foundation for machine agents acting with scoped, witnessed, revocable authority. The full specification is addressed in the agent extension.

Each extension takes PIC's categorical structure as its foundation. None requires modifying that structure. The model is complete; the extensions are applications of it.

8.5 Closing Statement

Physics abandoned anonymous state spaces when it encountered quantum systems. Computer science must now do the same for value. Just as quantum mechanics requires state to be indexed to the measurement context rather than stored in an anonymous global space, value computation requires state to be indexed to the participant. Both domains exhibit linear structure. Both prohibit copying. Both resolve through constituent participation rather than external reconciliation. This parallel is not metaphorical. It is categorical.

The K-Machine identified WHO as the missing dimension of computation — absent from every model since Turing, absent from every system since Shannon. PIC is what happens when WHO is present: not as a label, not as an access control identifier, but as a fundamental dimension of computation.

The result is not a better version of existing systems. Contextual Compute is not a faster distributed system. PIC is not a new agreement protocol. The Contextual Compute Protocol is not an optimisation of existing infrastructure. Together they constitute the formal foundation of a value-native computational stack.

$$\mathbf{C}_{\text{Cartesian}} \subsetneq V_{\text{PIC}}$$

The inclusion is strict. Classical computation is the degenerate case. Value computation — participant-indexed, linear, witnessed, composable, structurally commutative — is the general theory.

The correct model does not compete with incorrect ones. It makes their problems undefined. This is not a roadmap. It is a correction.

References

[1] Samson Abramsky and Bob Coecke. A categorical semantics of quantum protocols. In *Proceedings of the 19th Annual IEEE Symposium on Logic in Computer Science (LICS)*, pages 415–425. IEEE, 2004.

[2] John C. Baez and Mike Stay. *Physics, Topology, Logic and Computation: A Rosetta Stone*, volume 813 of *Lecture Notes in Physics*. 2011.

[3] Sam Blackshear, Evan Cheng, David L. Dill, et al. Move: A language with programmable resources. Libra Association White Paper, 2019.

[4] Sam Blackshear, Todd Nowacki, Dario Sezer, et al. The Sui smart contracts platform. Mysten Labs White Paper, 2023.

[5] Vitalik Buterin. A next-generation smart contract and decentralised application platform. Ethereum White Paper, 2014.

[6] Miguel Castro and Barbara Liskov. Practical byzantine fault tolerance. In *Proceedings of the 3rd USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 173–186. USENIX Association, 1999.

[7] Manuel M. T. Chakravarty, James Chapman, Kenneth MacKenzie, Orestis Melkonian, Michael Peyton Jones, and Philip Wadler. The extended UTxO model. IOHK Technical Report, 2020.

[8] Bob Coecke and Aleks Kissinger. *Picturing Quantum Processes: A First Course in Quantum Theory and Diagrammatic Reasoning*. Cambridge University Press, 2017.

[9] Jean-Yves Girard. Linear logic. *Theoretical Computer Science*, 50(1):1–101, 1987.

[10] Anantha Krishnan. Contextual compute: The general theory of computation. Zenodo, 2026. Contextual Compute Series, CC3. `https://doi.org/10.5281/zenodo.19177337`.

[11] Anantha Krishnan. On the structure of value: Value, information, and the cartesian degeneracy. Zenodo, 2026. Contextual Compute Series, CC2. `https://doi.org/10.5281/zenodo.19148907`.

[12] Anantha Krishnan. On the truth of value: The quantum value paper. Contextual Compute Series, CC1. Sarva Labs Working Paper, 2026.

[13] Karl Mazurak, Jianzhou Zhao, and Steve Zdancewic. Lightweight linear types in System F°. In *Proceedings of the 5th ACM SIGPLAN Workshop on Types in Language Design and Implementation (TLDI)*, pages 77–88. ACM, 2010.

[14] Satoshi Nakamoto. Bitcoin: A peer-to-peer electronic cash system, 2008.

[15] Claude E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27(3):379–423, 1948.

[16] Alexander Spiegelman, Neil Giridharan, Alberto Sonnino, and Lefteris Kokoris-Kogias. FastPay: High-performance byzantine fault tolerant settlement. In *Proceedings of the 4th ACM Conference on Advances in Financial Technologies (AFT)*, pages 163–177. ACM, 2022.

[17] Philip Wadler. Linear types can change the world! In Manfred Broy and Cliff B. Jones, editors, *Programming Concepts and Methods*, pages 546–566. North-Holland, 1990.

[18] David Walker. Substructural type systems. In Benjamin C. Pierce, editor, *Advanced Topics in Types and Programming Languages*, pages 3–43. MIT Press, 2005.